Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Comparing two solutions to the same problem

Today
Welcome to your next lesson: Comparing two solutions to the same problem.

I'm preparing your personalized lesson based on what you've learned, what comes next, and the best way to help you grow.

Step by step, you’re moving forward.

Let’s warm up with a 20-second recap using simple pictures in your mind 🧠

O(1): constant time → like checking 1 item on a shelf, no matter how long the shelf is.
O(n): linear time → walk down the shelf and check every item once.
O(n²): quadratic time → for every item on the shelf, you walk the whole shelf again (a loop inside a loop).

So the key idea is:

More nested loops over the same data → usually closer to O(n²).

One simple loop over the data → usually O(n).

A fixed amount of work → O(1).

Quick check before we compare real code:

If you see a Python function with one for loop that goes over a list once, what time complexity does it most likely have?

You can answer in words, like:

“constant”
“linear”
“quadratic”
linear

Exactly — linear ☑

One loop over n items → time grows roughly like n → O(n).

Now let’s compare two real Python solutions for the same task.

Task: sum all even numbers in a list.

Solution A (one loop):

Python

```
def sum_evens_a(nums):
    total = 0
    for x in nums:
        if x % 2 == 0:
            total += x
    return total
```

Solution B (nested loop over same list):

Python

```
def sum_evens_b(nums):
    total = 0
    for x in nums:
        for y in nums:
            if x == y and y % 2 == 0:
                total += y
    return total
```

Both give the same result.

Step 1: just by shape, which one do you expect to be faster for large lists: A or B?

a

What do you think? Take your time with the question below 👇

For a list of length n, how many times does the inner if line run in Solution A?

```
def sum_evens_a(nums):
    total = 0
    for x in nums:
        if x % 2 == 0:
            total += x
    return total
```

A.

About n times (once per element)

B.

About n^2 times

C.

A fixed number of times, not depending on n

Nice work, Ron 🎯 You picked the right answer.

In Solution A, the if x % 2 == 0: line is checked once for each element in nums.

So if the list has n elements:

The for x in nums: loop runs n times
Each time, Python reaches that if once
So the if line runs about n times → O(n)
Ready to zoom in on Solution B next and compare how many times its if runs?

Yes, let’s compare them

What do you think about Solution B? Pick an option below 👇

Now look at Solution B:

```
def sum_evens_b(nums):
    total = 0
    for x in nums:
        for y in nums:
            if x == y and y % 2 == 0:
                total += y
    return total
```

For a list of length n, how many times does the if x == y and y % 2 == 0: line get checked (run) in total?

A.

About n times

B.

About n² times (n multiplied by n)

C.

A fixed number of times, not depending on n

D.

It depends on how many even numbers are in the list

Nice, Ron — that was spot on again 👏

In Solution B, for a list of length n:

The outer loop runs n times (one for each x)
For each x, the inner loop runs n times (one for each y)
So total checks of
if x == y and y % 2 == 0:
≈ n * n = n² → O(n²)

So A is O(n) and B is O(n²) for the same task.

Next, I want to show you a simple race card table comparing A vs B (speed vs readability).

Are you okay if we keep using this sum evens example for that?

Yes, show the race card

Here’s your race card 🏁 comparing the two solutions side by side.

Key things to notice as you look at it:

For n = 4, Solution A does 4 if checks, Solution B does 16
As n grows, A grows like n, B grows like n²
Both solve the same problem, but A is shorter, clearer, and faster
Quick check, just in words:

If you had a list with 1,000,000 numbers, which solution would you pick in real code, and why?

You don’t need fancy terms — just “I’d pick X because …”

Race card: Comparing two algorithms for summing even numbers

I'd pick A because it's faster

Perfect reason, and exactly what a real engineer would say ☑

Now let's add one more real□world twist: sometimes the faster solution isn't always the one you pick — if the input is tiny, the simpler to read solution can be "good enough".

Imagine this:

You only ever process 5–10 numbers at a time.
Solution B was a little easier to understand for your teammate (even though it's slower for big lists).

In that situation, which would you choose for these always□small lists, and why: A or B?

No pressure — I just want your reasoning in plain language ☺

B for it simplicity and easier for future coders to modify and undertand the logic behind

Love that reasoning, Ron — that's the exact trade-off I wanted you to see 🤝

You've just described a key idea:

When inputs are small, a slightly slower but clearer solution can be the better choice, because humans reading the code matter too.

Let's lock this in with one more concrete scenario 👇

Two options again:

Solution A — O(n) (one loop), a bit more "technical"
Solution B — O(n²) (nested loops), but let's say it's super easy for beginners to read

Scenario:

You need to process every transaction in a 1,000,000□row log file every night.

Which do you choose here, and why — A or B?

B as its a very large ammount of data and the cpu workload will make it too slow to get output with A

Nice that you're thinking about CPU load — that's exactly the right instinct 🔥

But here we have to be careful with the direction:

For huge data (like 1,000,000 rows), we want the algorithm that does less work as n grows.
Solution A is O(n) → work grows like n
Solution B is O(n²) → work grows like n * n → this explodes for big n

So for a very large amount of data, we actually want Solution A, because it does far fewer operations than B and will be much faster.

To make it super concrete:

With n = 1,000,000
A: ~1,000,000 checks
B: ~1,000,000,000,000 checks (one trillion)

That’s why for big logs, we must care about time complexity.

Want to try a quick multiple‑choice to decide between A vs B in two different scenarios (tiny list vs huge list)?

yeah misread srrory i o get the nested is better

No worries at all, Ron — catching that is exactly how this stuff sticks 👍

You’ve got the key idea:

Nested loop → more work → usually slower for large n
Single loop → less work → usually better for big data

Let’s do this step by step with a clear picture so it’s rock solid:

For small tasks (like 5–10 items):
You can choose the one that’s easier to read, even if it’s a bit slower.
For huge tasks (like 1,000,000 rows):
You should choose the one with fewer steps as n grows → here, the single-loop, O(n) version.
To check this trade‑off in both directions (tiny vs huge lists), want to do that quick multiple‑choice comparison now?

Yes, give me the comparison

What do you think? Pick the pairing that feels right 👇

You have the same two solutions (A = O(n), B = O(n^2)) for summing evens in a list.

Scenario 1: Today’s shopping list with 10 items. Which solution is the more reasonable choice here?

Scenario 2: A big server log with 1,000,000 entries. Which solution is the more reasonable choice here?

Pick the best pairing:

A.

Scenario 1 → A, Scenario 2 → A

B.

Scenario 1 → B, Scenario 2 → B

C.

Scenario 1 → B, Scenario 2 → A

D.

Scenario 1 → A, Scenario 2 → B

Good catch to look at both scenarios together — this is a subtle one 👍

The best pairing is:

Scenario 1 (10 items) → B is reasonable

List is tiny, so even $O(n^2)$ is still very fast in practice.
If B were a bit clearer to read, we could choose it for readability.
Scenario 2 (1,000,000 entries) → A is the only sane choice

$O(n)$ → about 1,000,000 checks
$O(n^2)$ → about 1,000,000,000,000 checks (one trillion) — way too slow.

So:

Small input → readability can win (B ok).

Huge input → time complexity must win (A needed).

To make sure this really clicks, want to do one more quick comparison with a different story (like checking game scores or sensor data), or do you feel ready to summarize the rule in your own words?

Try a different story

Love it — new story coming up 🎮

Imagine you're writing code for a video game that tracks players' scores.

You have two solutions to find the total score of a player across many matches:

Solution A ($O(n)$):
You go through the list of that player's scores once and add them up.

Solution B ($O(n^2)$):
You use a nested loop: for each score, you loop over all scores again and add when indexes match (so you end up doing way more checks).

Now two situations:

Debug tool on your own PC

You only ever load at most 20 matches for testing.
Other developers might read this code; clarity is nice.
Live game server with millions of players

Each player can have hundreds of matches, and this runs constantly on the server.
Step by step:

For the debug tool (small: ~20 scores), which matters more:
raw speed, or simple/clear code?

For the live server (big: hundreds per player, millions of players), which matters more:
raw speed, or simple/clear code?

Answer in plain language first (no A/B yet):

What would you prioritize in each case?

for the debug tool simple clear code for the live server raw speed

Exactly right ☑

For the debug tool (tiny data) → simple, clear code is fine, even if it’s a bit slower.
For the live server (huge data, runs a lot) → raw speed (better time complexity) matters much more.

You’ve just nailed the core rule:

When inputs are small and rare, choose readability if you want.

When inputs are large or frequent, choose better time complexity.

Let’s tie it back to our A vs B idea and check mastery:

In your own words:

How would you decide between O(n) and O(n²) solutions when you face a new problem?

Just one or two sentences is enough.

Analyyze the current and futur potential data that will be input throug hhe algo. If it small and wil rmain so, go with easy to follow eve if its a bit ore cpu intensive but if it now or will gow i the guture to be a large data set then esig a tight efficient linear type algo

Ron, that’s a really solid answer 👏

You’ve got the key decision rule:

Look at how big the data is now and how big it might get later.

If it’s small and will stay small → you can choose the clearer solution, even if it’s a bit more CPU-heavy.
If it’s big now or will grow a lot → design a tighter, efficient (O(n)) algorithm.

That’s exactly how pros think about trade□offs.

Last piece for this lesson is to apply that choice directly to our A vs B code one more time, in a fresh scenario, to fully lock it in.

Want to do one final quick A□vs□B choice, or do you feel done and ready to move to the next lesson after a short recap?

One final A-vs-B choice

Take your time and pick the option that feels most reasonable for that weekly report tool 👇

You have the same two algorithms again:

A: One loop over the data → O(n)
B: Nested loops over the same data → O(n²)
New situation: You’re writing a weekly report tool that:

Reads between 5 and 30 records each week (and this won’t grow much)
Is run by you only, a few times per week
Will be shared later with new interns who are just learning Python
Which is the better overall choice here? Think about both speed and readability.

A.

Pick A: O(n), because it’s faster and the small extra complexity is fine here.

B.

Pick B: O(n²), because the data is small and we care more about super-clear code for interns.

C.

Either is fine, but B is slightly better because readability matters more than performance here.

You’re very close on that last one, Ron 👀

The “best” answer was:

Either is fine, but B is slightly better because readability matters more than performance here.

Why?

Data size is tiny (5–30 records, a few times a week) → performance difference between O(n) and O(n²) is basically invisible in practice.
The code will be read by new interns → super-clear code has extra value.
A (O(n)) is still objectively better algorithmically, but here the human factor dominates: clarity > micro□performance.

So in this very specific scenario:

A is technically better in complexity terms.
B can be slightly better overall if it really is clearer, because the data is small and speed doesn’t matter.

And you were already thinking in that direction — you picked B because you focused on interns and clarity, which is exactly the right instinct 🎯

You’ve now:

Compared two algorithms solving the same task using step counts and O(n) vs O(n²)
Talked through readability vs speed trade□offs
Chosen which algorithm to use in different real□world□style scenarios and explained why

From my side, that’s full mastery for this lesson.

Do you feel ready to wrap up this lesson and move to the next one, or do you want one more quick check question before we finish?

Wrap up and move on

Here’s your quick wrap-up, Ron 🎓

What you mastered today:

You compared two Python solutions that do the same job and judged them using:
Step counts for small n
Big O: O(1), O(n), O(n²)
You saw how:

One loop → usually O(n)
Nested loops on the same data → usually O(n²)
You learned a practical rule for trade□offs:
Small, rare tasks → readability can win
Large or frequent tasks → better time complexity must win

Decision rule you came up with (nicely put):

Look at current and future data size:
If it’s small and will stay small → clearer code is ok, even if a bit slower
If it’s big or will grow → design an efficient, linear-time style algorithm
That’s exactly how real engineers think when picking between two solutions. 🔥

Next up, we’ll build on this and keep sharpening your algorithm instincts step by step.

Before you jump ahead: anything from this lesson you want me to restate in a simpler way? If not, you’re good to hit Next Lesson and keep goin